ALEXANDRE Ewan, Groupe 1

03/11/2026

TP Hacking sans installation : Divulgation de fichiers sensibles

Table des matières

1) Problématique 1

2) Exercices 1

Exercice 1 : Analyse des fichiers robots.txt et exploitation de backup 1

Exercice 2 : Reconnaissance d'endpoints et divulgation d'informations sur une page de debug 4

3) Conclusion 5

1) Problématique

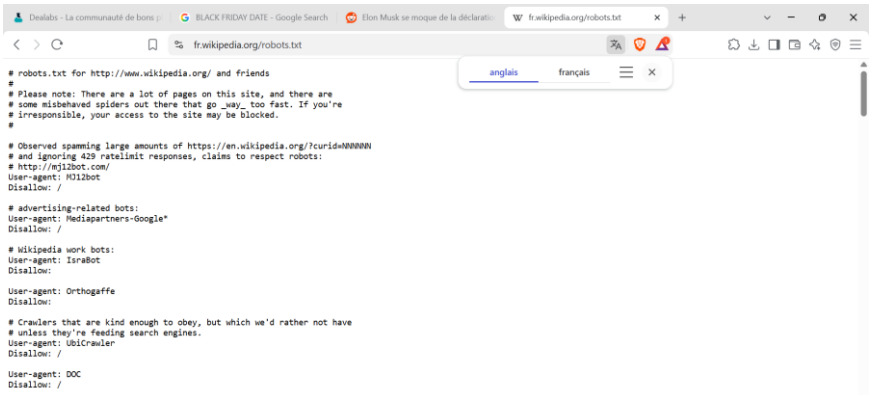
Nous allons voir durant ce compte rendu comment la divulgation de fichiers sensibles permet de découvrir des informations confidentielles sur une application web sans utiliser aucun outil complexe, simplement à l'aide de notre navigateur web. Nous analyserons le rôle et les limites des fichiers d'indexation comme robots.txt, puis nous verrons comment des oublis de déploiement (fichiers de sauvegarde .bak, endpoints d'administration, pages de diagnostic phpinfo) exposent des secrets critiques tels que des mots de passe de base de données ou des clés d'environnement.

2) Exercices

Exercice 1 : Analyse des fichiers robots.txt et exploitation de backup

1) Aller sur le site de Wikipedia et afficher le contenu du fichier robots.txt

On commence par ouvrir notre navigateur pour consulter le fichier robots.txt de Wikipédia :



2) Analyse des directives User-agent et Disallow

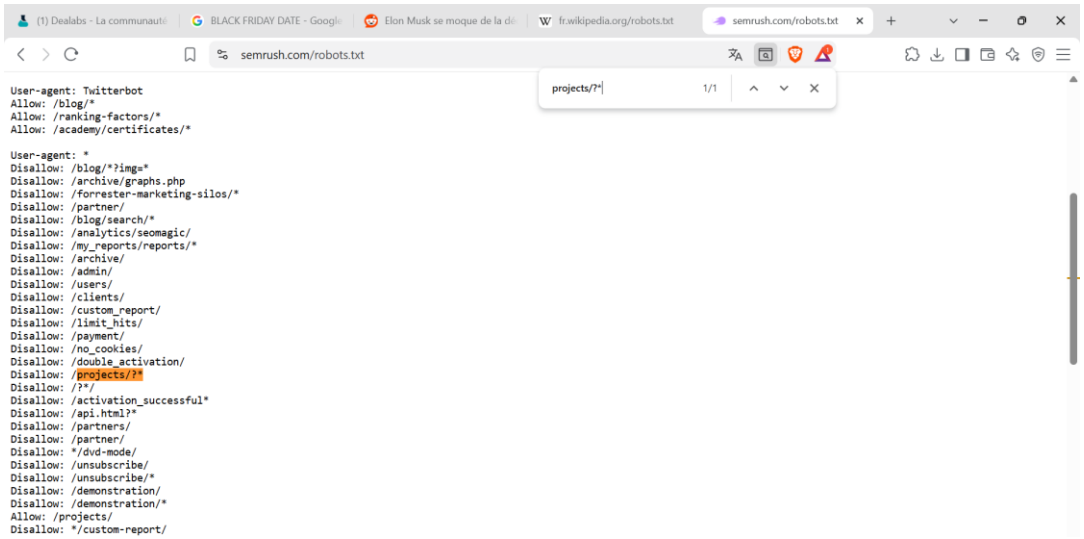
Question : Quelle différence y a-t-il entre :

User-agent: DOC
Disallow: /
et
User-agent: IsraBot
Disallow:

- Pour User-agent: DOC, la directive Disallow: / indique une interdiction totale d'accéder au site web (le slash représente la racine du domaine, donc l'ensemble de ses pages et répertoires).
- Pour User-agent: IsraBot, la directive Disallow: est laissée vide, ce qui signifie qu'aucune restriction d'exploration ne lui est appliquée. Le robot est donc pleinement autorisé à explorer et indexer l'intégralité du site.

3) Afficher le contenu du fichier robots.txt du site https://www.semrush.com

On se rend ensuite sur l'adresse du robots.txt de Semrush dans le navigateur :



4) Analyse du dossier /projects/ sur Semrush

Question : Le dossier /projects/ est-il globalement autorisé à l'exploration ? Quelle instruction le montre ?

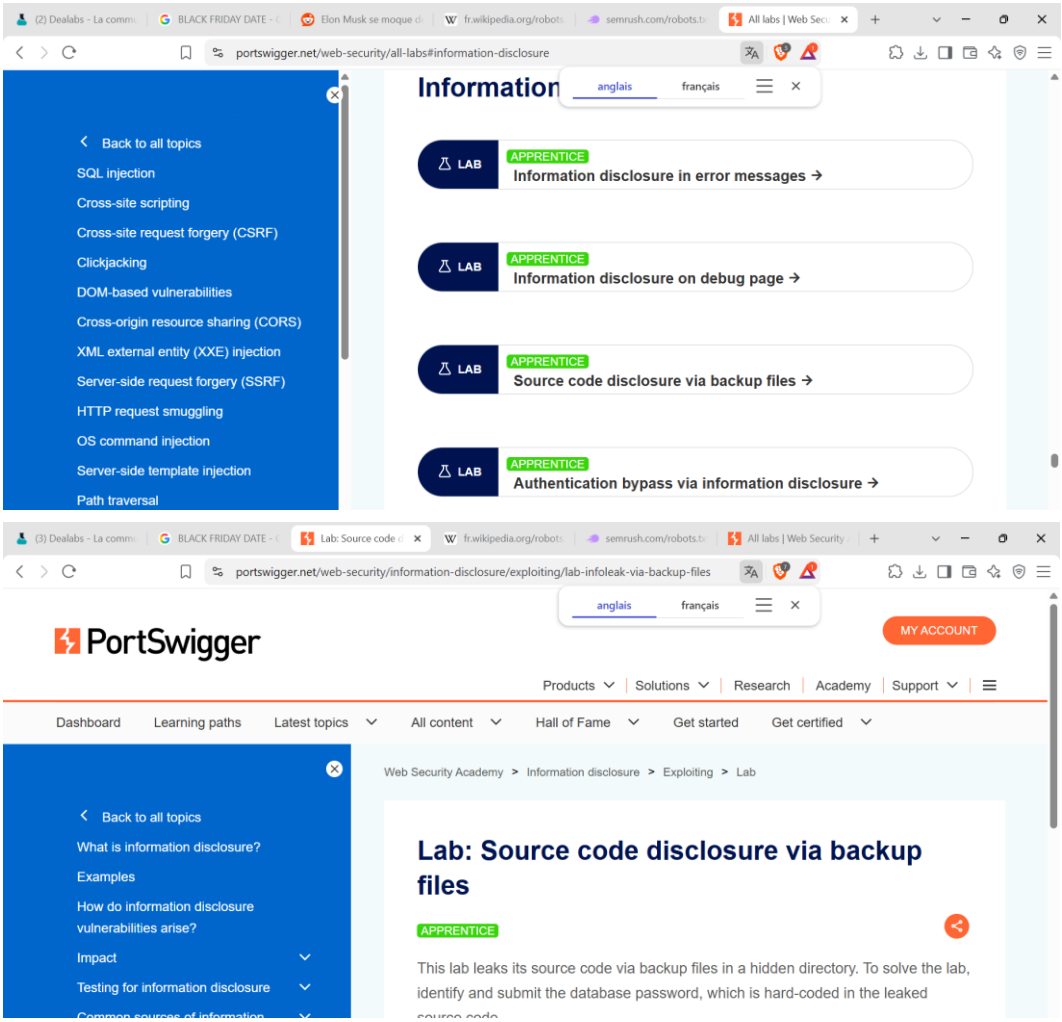
Oui, le dossier /projects/ est globalement autorisé à l'exploration pour les moteurs de recherche. C'est l'instruction suivante présente dans le fichier qui le démontre :
Allow: /projects/

Question : Expliquer : Disallow: /projects/?*

Cette instruction interdit aux robots d'explorer et d'indexer toutes les pages du dossier /projects/ qui contiennent des paramètres de requête dans l'URL (symbolisés par le point d'interrogation ? suivi de n'importe quels caractères représentés par *). Cela évite que les moteurs de recherche n'indexent des filtres de tri, des sessions ou des requêtes dynamiques inutiles.

5) Résolution du lab PortSwigger : Source code disclosure via backup files

On accède à la plateforme PortSwigger Web Security Academy dans la catégorie Information disclosure pour lancer le laboratoire d'exploitation de fichiers de sauvegarde :



Démarche d'exploitation et résolution étape par étape :

- Étape 1 : Une fois sur la boutique en ligne du lab, on consulte son fichier robots.txt en saisissant /robots.txt dans la barre d'adresse. On découvre une directive confidentielle : Disallow: /backup.
- Étape 2 : On navigue directement vers l'URL /backup. La fonctionnalité de traversée de répertoire (Directory Listing) est active côté serveur et liste les fichiers disponibles, dont un fichier de code source sauvegardé : ProductTemplate.java.bak.
- Étape 3 : On clique sur le fichier pour afficher son contenu. En examinant le code Java, on repère la ligne de connexion à la base de données PostgreSQL contenant le mot de passe en clair (admin / mot de passe hardcodé).
- Étape 4 : On clique sur le bouton « Submit solution » en haut de la page, on colle le mot de passe récupéré et on valide.
- Le laboratoire est immédiatement validé et passe au statut « Solved » avec la bannière de félicitations.

Exercice 2 : Reconnaissance d'endpoints et divulgation d'informations sur une page de debug

1) Notion d'endpoint et cartographie d'une application

Dans l'architecture web, un endpoint (point d'entrée) désigne une URL ou un chemin précis exposé par un serveur permettant d'accéder à une ressource ou de déclencher une fonctionnalité de l'application.

En complément de robots.txt, le fichier sitemap.xml sert de véritable cartographie en listant de manière structurée l'ensemble des pages publiées par le domaine.

Pour auditer un serveur, on s'appuie sur la liste des points d'entrée standards recensés dans la fiche technique :

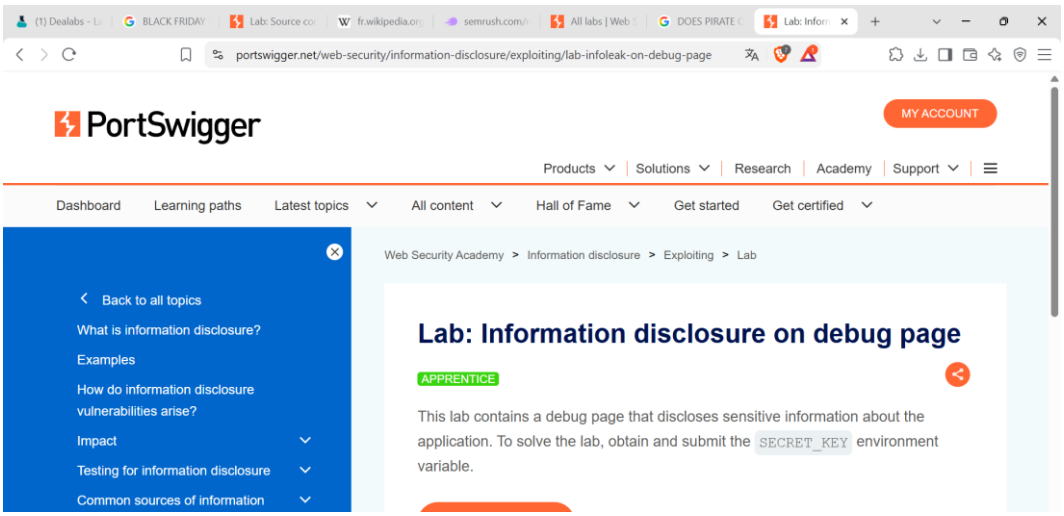
Points d'entrée / Chemins analysés	Description et risque de divulgation
robots.txt / sitemap.xml	Consignes d'indexation et cartographie des pages du site web.
/.env	Variables d'environnement (clés secrètes, tokens, mots de passe BDD).
/backup, /bak, /old	Fichiers et archives de sauvegarde contenant du code source obsolète ou sensible.
/cgi-bin/phpinfo.php, /info.php	Diagnostic complet de l'environnement PHP divulguant la configuration interne.
/admin, /administrator	Panneaux d'administration permettant la gestion et le contrôle de l'application.
/.git	Dépôt de contrôle de version exposant l'historique complet et les sources du projet.

2) Test d'endpoints standardisés et réalité du terrain

Lors de la phase de reconnaissance, tester des chemins théoriques comme /.env renvoie le plus souvent une réponse « Not Found » (code 404). Ce comportement est normal : une application n'est pas vulnérable par défaut, mais uniquement lorsqu'une erreur ou un oubli humain a été commis par l'équipe de développement lors de la mise en ligne.

3) Découverte de la page de debug et résolution du lab PortSwigger

On accède au laboratoire « Information disclosure on debug page » sur PortSwigger Web Security Academy :



Déroulement de la recherche et de la résolution :
- Étape 1 : En suivant la liste des points d'entrée bien connus (fichiers_sensibles_notes.pdf), on teste la présence d'interfaces de débogage dans les répertoires résiduels du serveur, notamment le dossier /cgi-bin/.
- Étape 2 : On saisit l'URL /cgi-bin/phpinfo.php dans la barre d'adresse du navigateur. Le serveur exécute le script et affiche la page complète de diagnostic phpinfo().
- Étape 3 : On effectue une recherche textuelle (Ctrl+F) sur le terme SECRET_KEY dans la page. On découvre la variable d'environnement SECRET_KEY divulguée en clair dans le tableau des variables PHP du serveur.
- Étape 4 : On copie la clé secrète obtenue, on clique sur « Submit solution » en haut de la boutique, on soumet la chaîne de caractères et on valide.
- Le laboratoire affiche la confirmation de réussite et passe au statut « Solved ».

4) Pourquoi la présence de phpinfo.php est-elle dangereuse ?
- Rôle d'origine : Le fichier phpinfo.php est un outil de diagnostic interne généré et employé par les développeurs pour vérifier l'environnement technique, les extensions activées et les paramètres de configuration pendant la conception.
- L'erreur de déploiement : L'équipe technique oublie fréquemment d'effacer ce script de test au moment du basculement de l'application vers l'environnement de production exposé au public.
- Impact sur la sécurité : La page expose l'intégralité des variables d'environnement (clés secrètes, identifiants BDD, chemins de répertoires système) ainsi que les versions exactes d'Apache, de PHP et du système d'exploitation. Ces informations stratégiques permettent à un attaquant de cibler précisément des vulnérabilités connues (CVE) pour compromettre le serveur.

5) Ce qu'il faut retenir (Bonnes pratiques défensives)
- Procédure de recette et de nettoyage : Avant tout passage en production, il est indispensable de contrôler systématiquement l'arborescence du serveur web afin de supprimer l'ensemble des scripts de test, fichiers de sauvegarde (.bak, .old), archives et outils de débogage.
- Gestion des secrets : Les clés d'API et les clés secrètes doivent être stockées dans des fichiers de configuration ou des gestionnaires de secrets situés strictement en dehors de la racine web publique du serveur (/var/www/html), afin d'être inaccessibles en consultation directe via un navigateur.

3) Conclusion

En conclusion, ce TP m'a permis de comprendre le fonctionnement de la divulgation de fichiers sensibles et l'importance de l'audit d'une application web sans aucun outil spécifique, simplement avec un navigateur. J'ai pu constater que les fichiers d'indexation comme robots.txt peuvent paradoxalement orienter un auditeur vers des zones masquées, et qu'un simple fichier de test ou de sauvegarde oublié en production peut divulguer des clés secrètes critiques compromettant l'ensemble du système.